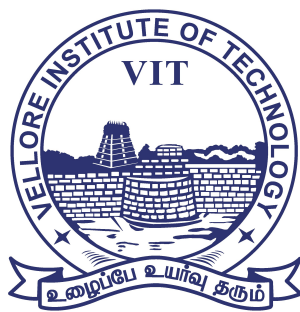




VIT[®]
B H O P A L
www.vitbhopal.ac.in

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothri kalan, Sehore

Madhya Pradesh - 466114

STUDENT MANAGEMENT SYSTEM USING PYTHON

**CSA1021 INTRODUCTION TO PROBLEM SOLVING
AND PROGRAMMING**

REG.NO : 26BAI10807

NAME : HARSH VARDHAN

BRANCH : B.Tech CSE (AIML)

SEMESTER: Fall Semester 2026-27

INDEX

Ex.NO	DATE	EXPERIMENT NAME	PAGE NO.
1	26/09/26	Abstract	1
2	26/09/26	Project Description	2
3	26/09/26	Introduction & Purpose of the Project	3
4	26/09/26	Problem Statement & Objectives	4
5	26/09/26	Scope of the Project	5
6	26/09/26	Student Management: View, Search, Update & Delete	6
7	26/09/26	Marks Management & Academic Performance Calculation	7
8	26/09/26	Performance Messages & Student Reports	8
9	26/09/26	JSON Data Storage & Validation / Error Handling	9
10	26/09/26	Program Architecture & Technologies Used	10
11	26/09/26	Source Code Appendix – Source Code Map	12
12	26/09/26	Execution Screenshots	25
13	26/09/26	Conclusion	31

01 • Abstract

A concise overview of the project, written for direct placement in the final report.

The **Student Management System using Python** is a console-based application developed to make the process of managing student information and academic performance easier, faster, and more organized. The main idea behind this project is to replace the traditional method of maintaining student records manually with a simple computerized system. Managing student details manually can become difficult when the number of records increases. It may lead to problems such as duplicate records, difficulty in finding information, mistakes while entering data, and errors while calculating academic performance. This project provides a simple solution to these problems by bringing the major student management activities together in one application.

The system allows the user to perform several important operations related to student records. A new student can be added by entering details such as student ID, name, course, and year or semester. Existing student records can be viewed, searched, updated, or deleted when required. The search facility allows the user to find a student using their student ID or name. The system also asks for confirmation before deleting a record, which helps prevent accidental deletion.

One of the important parts of the project is academic performance management. The system allows marks to be entered for four subjects: Python, Mathematics, Computer Fundamentals, and Communication Skills. Marks are accepted within the range of 0 to 100. After marks are entered, the application automatically calculates the total marks and percentage. Based on the percentage, a grade is assigned from A to F. The system also determines whether the student has passed or failed. A student must score at least 40 marks in every subject to be considered a pass. Along with the academic result, the system also provides a performance description such as Excellent Performance, Very Good Performance, Good Performance, Average Performance, or Needs Improvement.

The project uses **JSON-based storage** to save student information. This allows records to remain available even after the application is closed. JSON was selected because it is simple, lightweight, readable, and does not require a separate database system. The application also includes validation and error handling to make it more reliable. For example, duplicate student IDs are rejected, invalid names and empty course information are not accepted, and marks outside the valid range are rejected. The program also handles situations such as missing data files, invalid data and file-related errors.

The project has been developed using **Python and its standard library**, which makes it easy to run without installing additional external packages. It also provides an opportunity to understand important programming concepts such as variables, data types, conditional statements, loops, functions, dictionaries, file handling, JSON, exception handling, validation, and basic software design.

Overall, the Student Management System demonstrates how basic Python concepts can be combined to create a useful and practical application. Although the current system uses a simple console interface, it provides a strong foundation for future improvements such as a graphical user interface, database integration, login systems, attendance management, PDF report generation, data visualization, and cloud storage.

02 • Project Description

Detailed description of the project's purpose, scope, features, academic logic, storage, validation, architecture, technologies, learning outcomes and future scope.

1. Introduction

The **Student Management System using Python** is a practical software project developed to manage student records and academic information in a simple and organized manner. The project is designed as a console-based application, which means that all interaction takes place through the terminal or command prompt.

The main purpose of developing this system is to understand how Python programming can be used to solve a real-life problem. Instead of creating a program only to demonstrate individual concepts such as loops or functions, this project combines multiple concepts into one complete application.

In an educational environment, student information is continuously created, updated, searched, and reviewed. Details such as student names, identification numbers, courses, semesters, and marks need to be maintained properly. If this information is handled manually, managing it can become inconvenient and errors can occur. The Student Management System provides a simple digital alternative.

The application focuses on the basic requirements of a student management system while keeping the implementation suitable for a beginner-level Python project.

2. Purpose of the Project

The main purpose of the project is to provide an organized way to store and manage student information.

The application allows the user to perform different operations from a single menu. These operations include adding a student, viewing all students, searching for a particular student, updating information, deleting records, entering marks, and generating academic performance reports.

Another important purpose of the project is to demonstrate the practical use of Python programming concepts. The project uses functions, dictionaries, conditional statements, loops, file handling, JSON, validation, and exception handling.

Therefore, the project has two major purposes:

- To provide a simple student record management system.
- To provide practical experience in Python programming and application development.

3. Problem Statement

Managing student records manually can become difficult as the number of students increases. Finding a particular student's information may take time, while updating or deleting records manually can result in mistakes.

Another problem is academic performance calculation. If marks for several subjects have to be added and converted into percentages manually, there is always a possibility of calculation errors.

The Student Management System addresses these issues by providing a computerized method of storing student records and automatically processing academic information.

Instead of maintaining separate information manually, the user can perform the required operations through the application.

4. Objectives

The major objectives of this project are:

- To develop a practical application using Python.
- To understand how different Python concepts work together in a complete project.
- To digitally store student information.
- To provide basic CRUD operations.
- To search and retrieve student records easily.
- To allow modification and deletion of student information.
- To store and manage academic marks.
- To calculate total marks and percentage automatically.
- To assign grades according to predefined rules.
- To determine the pass or fail result.
- To generate student performance reports.
- To implement data validation.
- To handle common errors properly.
- To understand persistent data storage using JSON.

5. Scope of the Project

The current project focuses mainly on basic student and academic record management.

The system includes:

- Student registration
- Student record viewing
- Student searching
- Student information updating
- Student deletion
- Marks entry
- Marks validation
- Total calculation
- Percentage calculation
- Grade calculation
- Pass/fail determination
- Individual student reports
- Overall performance reports
- JSON-based data storage
- Input validation
- Error handling

The project is intentionally limited to a console-based environment so that the main focus remains on Python programming and logical implementation.

6. Student Management

Student management is the central part of the application.

When adding a student, the user is asked to provide a student ID, name, course, and year or semester. Before the information is stored, the system checks whether the entered information is valid.

The student ID must not be empty and should not contain spaces. The name is also checked to ensure that it contains appropriate characters. Course information cannot be left empty.

The system also checks whether the entered student ID already exists. If the ID is already present, the system does not create another record with the same ID.

This prevents duplicate student records and helps maintain consistency in the stored information.

7. Viewing Student Records

The application provides an option to view all students currently stored in the system.

The records are displayed in a structured format containing important information such as:

- Student ID
- Student Name
- Course
- Year/Semester

This makes it easier for the user to get an overview of the students currently present in the system.

If no student records are available, the system displays an appropriate message instead of showing an empty table.

8. Searching for Students

The search feature allows the user to find a particular student without manually checking every record.

The user can enter either a student ID or a student's name. The system then searches the stored records and displays matching students.

The search is case-insensitive, meaning that uppercase and lowercase differences do not prevent a match.

The system also supports partial matching. Therefore, the user does not necessarily have to enter the complete name or ID.

This makes the search process more convenient and user-friendly.

9. Updating Student Information

Student information may change over time. For example, a student may change their course or their academic year may be updated.

The Update Student feature allows the user to modify existing information.

The user first enters the student's ID. If the student exists, the current details are displayed as reference. The user can then enter new information.

An important feature is that the user does not have to re-enter every field. If a field is left blank, the existing information is retained.

This makes the update process simpler and avoids unnecessary data entry.

10. Deleting Student Records

The system also provides an option to delete a student record.

Deletion is treated carefully because it permanently removes information from the stored records. Before deleting the student, the application displays important student details and asks the user to confirm the operation.

The user can confirm the deletion or cancel it.

This small confirmation step helps reduce the possibility of accidentally deleting the wrong record.

11. Marks Management

Academic performance is an important part of the system.

The current application uses four subjects:

- Python
- Mathematics
- Computer Fundamentals
- Communication Skills

Each subject carries a maximum of 100 marks.

When the user enters marks, the system checks that the value is between 0 and 100. If the user enters an invalid number or a value outside the allowed range, the system displays an appropriate message and asks for valid input.

This prevents incorrect marks from being stored in the student record.

12. Academic Performance Calculation

After marks are entered, the application automatically calculates the student's academic performance.

The total marks are calculated by adding the marks obtained in all subjects.

For example:

Total Marks = Python + Mathematics + Computer Fundamentals + Communication Skills

The percentage is then calculated using the total marks and the maximum possible marks.

For four subjects:

Maximum Marks = 400

Therefore:

Percentage = (Total Marks / 400) × 100

The system then uses the percentage to assign a grade.

Percentage	Grade
90% and above	A
75% to below 90%	B
60% to below 75%	C
50% to below 60%	D

Percentage	Grade
Below 50%	F

The result is calculated separately. A student must score at least 40 marks in every subject to receive a Pass result.

13. Performance Messages

In addition to displaying the grade, the system provides a simple performance description.

Depending on the percentage, the application can display:

- Excellent Performance
- Very Good Performance
- Good Performance
- Average Performance
- Needs Improvement

This gives the report a more understandable interpretation rather than displaying only numerical values.

For example, a student with a percentage above 90% receives an "Excellent Performance" message.

14. Student Performance Report

The application can generate a detailed report for an individual student.

The report contains:

- Student ID
- Name
- Course
- Year/Semester
- Subject-wise marks
- Total marks
- Percentage
- Grade
- Result
- Performance message

This provides a complete overview of the student's academic information.

The application can also generate performance information for all students. This is useful when the user wants to compare or review the overall academic records stored in the system.

15. Data Storage Using JSON

The application uses JSON for storing student information.

JSON is a lightweight format that is easy for both programs and humans to understand. It also works well with Python because Python provides built-in support for reading and writing JSON data.

The use of JSON means that student information does not disappear when the program is closed.

The application loads existing information when it starts and saves updated information whenever student records are changed.

This provides basic persistent storage without requiring a separate database server.

16. Validation and Error Handling

Validation is an important part of the project because users may enter incorrect information.

The application checks different types of input before accepting them.

For example:

- Student ID cannot be empty.
- Student ID cannot contain spaces.
- Student names are validated.
- Course information cannot be empty.
- Marks must be between 0 and 100.
- Duplicate student IDs are rejected.
- Empty search values are rejected.
- Missing student records are handled.
- Invalid numerical input is handled safely.

The program also handles errors related to reading and writing the JSON file.

This improves the reliability of the application and prevents many common problems from terminating the program unexpectedly.

17. Program Architecture

The application follows a logical structure in which different responsibilities are handled by different functions.

The main program controls the overall flow and displays the menu. Depending on the user's selection, the program calls the appropriate function.

For example:

User → Main Menu → Selected Operation → Validation → Processing → Storage/Report → Output

The project can later be divided into separate modules for student management, marks, reports, validation, file handling, and menu control.

This would make the project easier to maintain and extend as it becomes larger.

18. Technologies Used

The project uses **Python 3** as its main programming language.

Other technologies and tools involved include:

- Python Standard Library
- JSON
- Git
- GitHub

- unittest

One advantage of the current implementation is that it does not require external Python packages. This keeps installation simple and allows the project to run in a normal Python environment.

19. Learning Outcomes

Developing this project provided practical experience with several Python concepts.

The major concepts involved include:

- Variables
- Data types
- Operators
- Conditional statements
- Loops
- Functions
- Lists
- Dictionaries
- File handling
- JSON
- Exception handling
- Input validation
- CRUD operations
- Basic application architecture
- Testing
- Git
- GitHub

The project also provides an understanding of how individual programming concepts can be combined to create a complete application.

20. Future Scope

Although the current application is console-based, it can be improved significantly in the future.

A graphical interface can be developed using **Tkinter**, allowing users to interact with buttons, forms and tables instead of a terminal.

JSON storage can be replaced with an **SQLite database** to provide more advanced data management.

A login system can also be introduced with different roles for administrators, teachers and students.

Other possible improvements include:

- Attendance management
- PDF report generation
- Data visualization
- Advanced search and filtering
- Cloud storage

- Additional academic subjects
- Student profile management
- Automated report generation

These improvements can gradually transform the project from a basic educational application into a more complete academic management system.

21. Overall Description

The Student Management System is a practical demonstration of how Python can be used to solve a real-world information-management problem. It provides the basic functionality required to maintain student records while also introducing academic performance analysis.

The project is simple enough to understand and demonstrate, but it contains enough functionality to show the practical use of programming concepts learned during the early stages of a B.Tech CSE/AIML course.

The most important aspect of the project is that it connects programming concepts with an actual use case. Instead of using separate examples for functions, dictionaries, file handling, validation and conditional logic, all of these concepts are brought together into one working system.

The project therefore provides a strong foundation for future development. It can begin as a simple console application and gradually evolve into a database-driven, graphical and cloud-connected student management platform.

Source Code Map

The implementation is presented as a single Python file, with responsibilities organized into clearly marked sections.

No.	Section	Key Responsibility
01	Data Storage Location	get_data_file(), DATA_FILE
02	Subjects	SUBJECTS configuration
03	File Handling	load_data(), save_data()
04	Validation Functions	ID, name, course and marks
05	Marks Functions	total, percentage, grade, result
06	Add Student	add_student()
07	View All Students	view_all_students()
08	Search Student	search_student()
09	Update Student	update_student()
10	Delete Student	delete_student()
11	Enter / Update Marks	enter_marks()
12	Generate Student Report	generate_student_report()
13	All Performance Reports	generate_all_reports()
14	Display Menu	display_menu()
15	Main Function	main()
16	Program Start	__name__ == '__main__'

The code is reproduced as supplied. Only extracted line markers used by the document viewer have been removed so the appendix reads as normal Python source.

1. Add Student

```
0001
0002 # =====
0003 # STUDENT MANAGEMENT SYSTEM
0004 # Python Essentials - Evaluated Course Project
0005 # =====
0006 #
0007 # Author: HARSH VARDHAN
0008 # Course: B.Tech CSE (AIML)
0009 #
0010 # SINGLE-FILE VERSION
0011 #
0012 # Features:
0013 # 1. Add Student
0014 # 2. View All Students
0015 # 3. Search Student
0016 # 4. Update Student
0017 # 5. Delete Student
0018 # 6. Enter / Update Marks
0019 # 7. Generate Student Report
0020 # 8. Generate All Performance Reports
0021 # 9. Exit
0022 #
0023 # FIX:
0024 # Student data is stored in a writable folder inside the
0025 # user's account instead of the project folder.
0026 # =====
0027
0028 import json
0029 import os
0030 from pathlib import Path
0031
0032 # =====
0033 # 1. DATA STORAGE LOCATION
0034 # =====
0035
0036 def get_data_file():
0037     """
0038     Creates a writable folder for storing student data.
0039
0040     On Windows:
0041         C:\\Users\\YourName\\StudentManagementSystemData\\
0042
0043     This avoids permission problems in protected folders.
0044     """
0045
0046     try:
0047         # Windows local application data folder
0048         local_app_data = os.environ.get("LOCALAPPDATA")
0049
0050         if local_app_data:
0051             data_folder = Path(local_app_data) / "StudentManagementSystem"
0052         else:
0053             # Fallback for other operating systems
0054             data_folder = Path.home() / "StudentManagementSystemData"
0055
0056             # Create folder if it does not exist
0057             data_folder.mkdir(parents=True, exist_ok=True)
0058
0059             return data_folder / "students.json"
0060
0061     except Exception:
0062         # Final fallback
0063         data_folder = Path.home() / "StudentManagementSystemData"
0064         data_folder.mkdir(parents=True, exist_ok=True)
0065
0066         return data_folder / "students.json"
0067
0068 DATA_FILE = get_data_file()
0069
0070 # =====
0071 # 2. SUBJECTS
0072 # =====
0073
0074 SUBJECTS = [
0075     "Python",
```

3. FILE HANDLING

```
0079     "Mathematics",
0080     "Computer Fundamentals",
0081     "Communication Skills"
0082 ]
0083
0084 # =====
0085 # 3. FILE HANDLING
0086 # =====
0087
0088 def load_data():
0089     """
0090     Load student data from JSON file.
0091
0092     If the file does not exist, an empty dictionary is returned.
0093     """
0094     try:
0095         if not DATA_FILE.exists():
0096             return {}
0097
0098         with open(DATA_FILE, "r", encoding="utf-8") as file:
0099             data = json.load(file)
0100
0101             # Make sure the loaded data is a dictionary
0102             if isinstance(data, dict):
0103                 return data
0104
0105             print("\nWarning: Invalid data format.")
0106             return {}
0107
0108     except json.JSONDecodeError:
0109         print("\nWarning: students.json contains invalid data.")
0110         print("Starting with an empty student database.")
0111
0112         return {}
0113
0114     except PermissionError:
0115         print("\nPermission error while reading student data.")
0116         print("Please make sure the file is not open in another program.")
0117
0118         return {}
0119
0120     except Exception as error:
0121         print(f"\nError loading data: {error}")
0122
0123         return {}
0124
0125 def save_data(data):
0126     """
0127     Save student data safely to JSON file.
0128
0129     A temporary file is used first. This reduces the chance
0130     of corrupting the main JSON file.
0131     """
0132     try:
0133         # Make sure the directory exists
0134         DATA_FILE.parent.mkdir(parents=True, exist_ok=True)
0135
0136         # Temporary file
0137         temp_file = DATA_FILE.with_suffix(".tmp")
0138
0139         # Write data to temporary file
0140         with open(temp_file, "w", encoding="utf-8") as file:
0141             json.dump(
0142                 data,
0143                 file,
0144                 indent=4,
0145                 ensure_ascii=False
0146             )
```

4. VALIDATION FUNCTIONS

```
0157
0158         # Make sure everything is written
0159         file.flush()
0160         os.fsync(file.fileno())
0161
0162         # Replace old file with new file
0163         os.replace(temp_file, DATA_FILE)
0164
0165         return True
0166
0167     except PermissionError:
0168
0169         print("\n" + "=" * 60)
0170         print("PERMISSION ERROR")
0171         print("=" * 60)
0172
0173         print("Python does not have permission to save the file.")
0174         print("\nTry these steps:")
0175         print("1. Close students.json if it is open.")
0176         print("2. Close the program.")
0177         print("3. Run the program again.")
0178         print("4. Make sure the project is not inside a protected folder.")
0179
0180         return False
0181
0182     except Exception as error:
0183
0184         print(f"\nError saving data: {error}")
0185
0186         return False
0187
0188
0189 # =====
0190 # 4. VALIDATION FUNCTIONS
0191 # =====
0192
0193 def validate_student_id(student_id):
0194     """
0195     Validate student ID.
0196     """
0197
0198     student_id = student_id.strip()
0199
0200     if student_id == "":
0201         return False
0202
0203     if " " in student_id:
0204         return False
0205
0206     return True
0207
0208
0209 def validate_name(name):
0210     """
0211     Validate student name.
0212
0213     Only alphabets and spaces are allowed.
0214     """
0215
0216     name = name.strip()
0217
0218     if name == "":
0219         return False
0220
0221     if not all(
0222         character.isalpha() or character.isspace()
0223         for character in name
0224     ):
0225         return False
0226
0227     return True
0228
0229
0230 def validate_course(course):
0231     """
0232     Validate course.
0233     """
0234
```

5. MARKS FUNCTIONS

```
0235     if course.strip() == "":
0236         return False
0237
0238     return True
0239
0240
0241 def get_valid_mark(subject):
0242     """
0243     Get valid marks between 0 and 100.
0244     """
0245
0246     while True:
0247
0248         try:
0249
0250             mark = float(
0251                 input(f"Enter marks for {subject} (0-100): ")
0252             )
0253
0254             if 0 <= mark <= 100:
0255                 return mark
0256
0257             print("Marks must be between 0 and 100.")
0258
0259         except ValueError:
0260
0261             print("Please enter a valid number.")
0262
0263
0264 # =====
0265 # 5. MARKS FUNCTIONS
0266 # =====
0267
0268 def calculate_total(marks):
0269     """
0270     Calculate total marks.
0271     """
0272
0273     return sum(marks.values())
0274
0275
0276 def calculate_percentage(marks):
0277     """
0278     Calculate percentage.
0279     """
0280
0281     if not marks:
0282         return 0
0283
0284     total = calculate_total(marks)
0285
0286     maximum_marks = len(marks) * 100
0287
0288     percentage = (total / maximum_marks) * 100
0289
0290     return percentage
0291
0292
0293 def calculate_grade(percentage):
0294     """
0295     Calculate grade.
0296
0297     90 and above = A
0298     75-89       = B
0299     60-74       = C
0300     50-59       = D
0301     Below 50    = F
0302     """
0303
0304     if percentage >= 90:
0305         return "A"
0306
0307     elif percentage >= 75:
0308         return "B"
0309
0310     elif percentage >= 60:
0311         return "C"
0312
```

6. ADD STUDENT

```
0313     elif percentage >= 50:
0314         return "D"
0315
0316     else:
0317         return "F"
0318
0319
0320 def calculate_result(marks):
0321     """
0322     Calculate Pass/Fail.
0323
0324     Minimum 40 marks are required in every subject.
0325     """
0326
0327     if not marks:
0328         return "Not Available"
0329
0330     for mark in marks.values():
0331         if mark < 40:
0332             return "Fail"
0333
0334     return "Pass"
0335
0336
0337 def performance_message(percentage):
0338     """
0339     Display performance message.
0340     """
0341
0342     if percentage >= 90:
0343         return "Excellent Performance"
0344
0345     elif percentage >= 75:
0346         return "Very Good Performance"
0347
0348     elif percentage >= 60:
0349         return "Good Performance"
0350
0351     elif percentage >= 50:
0352         return "Average Performance"
0353
0354     else:
0355         return "Needs Improvement"
0356
0357
0358
0359 # =====
0360 # 6. ADD STUDENT
0361 # =====
0362
0363 def add_student(data):
0364
0365     print("\n" + "=" * 60)
0366     print("                                ADD STUDENT")
0367     print("=" * 60)
0368
0369     # Student ID
0370     student_id = input("Enter Student ID: ").strip()
0371
0372     if not validate_student_id(student_id):
0373
0374         print("\nInvalid Student ID.")
0375         print("Student ID cannot be empty or contain spaces.")
0376
0377         return
0378
0379     # Check duplicate ID
0380     if student_id in data:
0381
0382         print("\nStudent ID already exists.")
0383
0384         return
0385
0386     # Student Name
0387     name = input("Enter Student Name: ").strip()
0388
0389     if not validate_name(name):
0390
```

7. VIEW ALL STUDENTS

```
0391         print("\nInvalid name.")
0392         print("Name should contain alphabets and spaces only.")
0393
0394         return
0395
0396     # Course
0397     course = input("Enter Course: ").strip()
0398
0399     if not validate_course(course):
0400
0401         print("\nCourse cannot be empty.")
0402
0403         return
0404
0405     # Year
0406     year = input("Enter Year/Semester: ").strip()
0407
0408     if year == "":
0409         year = "Not Specified"
0410
0411     # Create student
0412     student = {
0413
0414         "student_id": student_id,
0415
0416         "name": name,
0417
0418         "course": course,
0419
0420         "year": year,
0421
0422         "marks": {}
0423     }
0424
0425     # Add to dictionary
0426     data[student_id] = student
0427
0428     # Save
0429     if save_data(data):
0430
0431         print("\n" + "-" * 60)
0432         print("Student added successfully!")
0433         print("-" * 60)
0434
0435         print("Student ID :", student_id)
0436         print("Name      :", name)
0437         print("Course   :", course)
0438         print("Year     :", year)
0439
0440
0441
0442 # =====
0443 # 7. VIEW ALL STUDENTS
0444 # =====
0445
0446 def view_all_students(data):
0447
0448     print("\n" + "=" * 80)
0449     print("                                ALL STUDENTS")
0450     print("=" * 80)
0451
0452     if not data:
0453
0454         print("No students found.")
0455
0456         return
0457
0458     print(
0459         f"{'ID':<12}"
0460         f"{'Name':<25}"
0461         f"{'Course':<25}"
0462         f"{'Year':<15}"
0463     )
0464
0465     print("-" * 80)
0466
0467     for student in data.values():
0468
```

8. SEARCH STUDENT

```
0469         print(
0470             f"{student['student_id']:<12}"
0471             f"{student['name']:<25}"
0472             f"{student['course']:<25}"
0473             f"{student['year']:<15}"
0474         )
0475
0476
0477 # =====
0478 # 8. SEARCH STUDENT
0479 # =====
0480
0481 def search_student(data):
0482
0483     print("\n" + "=" * 60)
0484     print("                                SEARCH STUDENT")
0485     print("=" * 60)
0486
0487     search = input(
0488         "Enter Student ID or Name: "
0489     ).strip().lower()
0490
0491     if search == "":
0492
0493         print("Search cannot be empty.")
0494
0495         return
0496
0497     found = False
0498
0499     for student in data.values():
0500
0501         student_id = student["student_id"].lower()
0502
0503         student_name = student["name"].lower()
0504
0505         if search in student_id or search in student_name:
0506
0507             print("\nStudent Found")
0508             print("-" * 40)
0509
0510             print("Student ID :", student["student_id"])
0511             print("Name       :", student["name"])
0512             print("Course      :", student["course"])
0513             print("Year        :", student["year"])
0514
0515             found = True
0516
0517     if not found:
0518
0519         print("\nNo student found.")
0520
0521
0522 # =====
0523 # 9. UPDATE STUDENT
0524 # =====
0525
0526 def update_student(data):
0527
0528     print("\n" + "=" * 60)
0529     print("                                UPDATE STUDENT")
0530     print("=" * 60)
0531
0532     student_id = input("Enter Student ID: ").strip()
0533
0534     if student_id not in data:
0535
0536         print("\nStudent not found.")
0537
0538         return
0539
0540     student = data[student_id]
0541
0542     print("\nPress ENTER to keep the existing value.")
0543
0544     # Update name
0545     name = input(
0546         f"Enter new name [{student['name']}]: "
```

10. DELETE STUDENT

```
0547     ).strip()
0548
0549     if name != "":
0550         if validate_name(name):
0551             student["name"] = name
0552         else:
0553             print("Invalid name.")
0554             return
0555
0556     # Update course
0557     course = input(
0558         f"Enter new course [{student['course']}]: "
0559     ).strip()
0560
0561     if course != "":
0562         if validate_course(course):
0563             student["course"] = course
0564         else:
0565             print("Invalid course.")
0566             return
0567
0568     # Update year
0569     year = input(
0570         f"Enter new year/semester [{student['year']}]: "
0571     ).strip()
0572
0573     if year != "":
0574         student["year"] = year
0575
0576     # Save
0577     if save_data(data):
0578         print("\nStudent updated successfully!")
0579
0580
0581 # =====
0582 # 10. DELETE STUDENT
0583 # =====
0584
0585 def delete_student(data):
0586
0587     print("\n" + "=" * 60)
0588     print("                                DELETE STUDENT")
0589     print("=" * 60)
0590
0591     student_id = input("Enter Student ID: ").strip()
0592
0593     if student_id not in data:
0594         print("\nStudent not found.")
0595         return
0596
0597     student = data[student_id]
0598
0599     print("\nStudent Details")
0600     print("-" * 40)
0601
0602     print("Student ID :", student["student_id"])
0603     print("Name       :", student["name"])
0604     print("Course      :", student["course"])
0605
0606     confirm = input(
0607         "\nAre you sure you want to delete this student? (y/n): "
0608     ).strip().lower()
0609
0610     if confirm == "y":
0611         del data[student_id]
```

11. ENTER / UPDATE MARKS

```
0625         if save_data(data):
0626
0627             print("\nStudent deleted successfully!")
0628
0629         else:
0630
0631             print("\nDeletion cancelled.")
0632
0633
0634 # =====
0635 # 11. ENTER / UPDATE MARKS
0636 # =====
0637
0638 def enter_marks(data):
0639
0640     print("\n" + "=" * 60)
0641     print("                                ENTER / UPDATE MARKS")
0642     print("=" * 60)
0643
0644     student_id = input("Enter Student ID: ").strip()
0645
0646     if student_id not in data:
0647
0648         print("\nStudent not found.")
0649
0650         return
0651
0652     student = data[student_id]
0653
0654     print("\nStudent:", student["name"])
0655
0656     print("-" * 40)
0657
0658     marks = {}
0659
0660     for subject in SUBJECTS:
0661
0662         marks[subject] = get_valid_mark(subject)
0663
0664     student["marks"] = marks
0665
0666     if save_data(data):
0667
0668         print("\nMarks saved successfully!")
0669
0670
0671 # =====
0672 # 12. GENERATE STUDENT REPORT
0673 # =====
0674
0675 def generate_student_report(data):
0676
0677     print("\n" + "=" * 60)
0678     print("                                STUDENT REPORT")
0679     print("=" * 60)
0680
0681     student_id = input("Enter Student ID: ").strip()
0682
0683     if student_id not in data:
0684
0685         print("\nStudent not found.")
0686
0687         return
0688
0689     student = data[student_id]
0690
0691     print("\n")
0692     print("=" * 60)
0693     print("                                STUDENT PERFORMANCE REPORT")
0694     print("=" * 60)
0695
0696     print("Student ID :", student["student_id"])
0697     print("Name       :", student["name"])
0698     print("Course    :", student["course"])
0699     print("Year      :", student["year"])
0700
0701     print("\nMarks")
0702     print("-" * 60)
```

13. ALL PERFORMANCE REPORTS

```
0703
0704     marks = student["marks"]
0705
0706     if not marks:
0707
0708         print("Marks have not been entered yet.")
0709
0710         print("-" * 60)
0711
0712         return
0713
0714     for subject, mark in marks.items():
0715
0716         print(
0717             f"{subject:<35} : {mark:.2f}"
0718         )
0719
0720     total = calculate_total(marks)
0721
0722     percentage = calculate_percentage(marks)
0723
0724     grade = calculate_grade(percentage)
0725
0726     result = calculate_result(marks)
0727
0728     performance = performance_message(percentage)
0729
0730     print("-" * 60)
0731
0732     print(f"Total Marks : {total:.2f}")
0733     print(f"Percentage : {percentage:.2f}%")
0734     print(f"Grade : {grade}")
0735     print(f"Result : {result}")
0736     print(f"Performance : {performance}")
0737
0738     print("-" * 60)
0739
0740
0741 # =====
0742 # 13. ALL PERFORMANCE REPORTS
0743 # =====
0744
0745 def generate_all_reports(data):
0746
0747     print("\n" + "-" * 85)
0748     print("                ALL PERFORMANCE REPORTS")
0749     print("-" * 85)
0750
0751     if not data:
0752
0753         print("No students found.")
0754
0755         return
0756
0757     print(
0758         f"{'ID':<12}"
0759         f"{'Name':<25}"
0760         f"{'Percentage':<15}"
0761         f"{'Grade':<10}"
0762         f"{'Result':<10}"
0763     )
0764
0765     print("-" * 85)
0766
0767     for student in data.values():
0768
0769         marks = student["marks"]
0770
0771         if marks:
0772
0773             percentage = calculate_percentage(marks)
0774
0775             grade = calculate_grade(percentage)
0776
0777             result = calculate_result(marks)
0778
0779             print(
0780                 f"{student['student_id']:<12}"
```

14. DISPLAY MENU

```
0781             f"{student['name']:<25}"
0782             f"{percentage:<15.2f}"
0783             f"{grade:<10}"
0784             f"{result:<10}"
0785         )
0786
0787     else:
0788
0789         print(
0790             f"{student['student_id']:<12}"
0791             f"{student['name']:<25}"
0792             f"{ 'Not Available':<15}"
0793             f"{'-':<10}"
0794             f"{'-':<10}"
0795         )
0796
0797     print("-" * 85)
0798
0799
0800 # =====
0801 # 14. DISPLAY MENU
0802 # =====
0803
0804 def display_menu():
0805
0806     print("\n")
0807     print("=" * 60)
0808     print("          STUDENT MANAGEMENT SYSTEM")
0809     print("=" * 60)
0810
0811     print("1. Add Student")
0812     print("2. View All Students")
0813     print("3. Search Student")
0814     print("4. Update Student")
0815     print("5. Delete Student")
0816     print("6. Enter / Update Marks")
0817     print("7. Generate Student Report")
0818     print("8. Generate All Performance Reports")
0819     print("9. Exit")
0820
0821     print("=" * 60)
0822
0823
0824 # =====
0825 # 15. MAIN FUNCTION
0826 # =====
0827
0828 def main():
0829
0830     # Load existing student data
0831     data = load_data()
0832
0833     print("\n")
0834     print("=" * 60)
0835     print("          WELCOME TO STUDENT MANAGEMENT SYSTEM")
0836     print("=" * 60)
0837
0838     print("\nData file location:")
0839     print(DATA_FILE)
0840
0841     while True:
0842
0843         display_menu()
0844
0845         choice = input(
0846             "Enter your choice (1-9): "
0847         ).strip()
0848
0849         # -----
0850         # OPTION 1
0851         # -----
0852
0853         if choice == "1":
0854
0855             add_student(data)
0856
0857         # -----
0858         # OPTION 2
```

```
0859 # -----
0860
0861 elif choice == "2":
0862     view_all_students(data)
0863
0864 # -----
0865 # OPTION 3
0866 # -----
0867
0868 elif choice == "3":
0869     search_student(data)
0870
0871 # -----
0872 # OPTION 4
0873 # -----
0874
0875 elif choice == "4":
0876     update_student(data)
0877
0878 # -----
0879 # OPTION 5
0880 # -----
0881
0882 elif choice == "5":
0883     delete_student(data)
0884
0885 # -----
0886 # OPTION 6
0887 # -----
0888
0889 elif choice == "6":
0890     enter_marks(data)
0891
0892 # -----
0893 # OPTION 7
0894 # -----
0895
0896 elif choice == "7":
0897     generate_student_report(data)
0898
0899 # -----
0900 # OPTION 8
0901 # -----
0902
0903 elif choice == "8":
0904     generate_all_reports(data)
0905
0906 # -----
0907 # OPTION 9
0908 # -----
0909
0910 elif choice == "9":
0911     print("\n" + "=" * 60)
0912     print("Thank you for using Student Management System!")
0913     print("Goodbye!")
0914     print("=" * 60)
0915
0916     break
0917
0918 # -----
0919 # INVALID OPTION
0920 # -----
0921
0922 else:
0923     print("\nInvalid choice!")
0924     print("Please enter a number from 1 to 9.")
0925
0926 # =====
0927
```

```
0937 # PROGRAM START
0938 # =====
0939
0940 if __name__ == "__main__":
0941     main()
0942
0943
0944
0945 #created BY HARSH VARDHAN
0946
```

13. EXECUTION SCREENSHOTS

```
=====
                        STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Enter / Update Marks
7. Generate Student Report
8. Generate All Performance Reports
9. Exit
=====
```

Figure 1. Main Menu

```
=====
Enter your choice (1-9): 1
=====
                        ADD STUDENT
=====
Enter Student ID: 26BAI10807
Enter Student Name: HARSH VARDHAN
Enter Course: B.Tech CSE (AIML)
Enter Year/Semester: 1st SEMESTER

-----
Student added successfully!
-----

Student ID : 26BAI10807
Name       : HARSH VARDHAN
Course     : B.Tech CSE (AIML)
Year       : 1st SEMESTER
```

Figure 2. Add Student

Enter your choice (1-9): 2

ALL STUDENTS

ID	Name	Course	Year
26BAI10807	HARSH VARDHAN	B.Tech CSE (AIML)	1st SEMESTER

Figure 3. View All Students

```
=====
Enter your choice (1-9): 3

=====
                        SEARCH STUDENT
=====
Enter Student ID or Name: HARSH VARDHAN

Student Found
-----
Student ID : 26BAI10807
Name       : HARSH VARDHAN
Course     : B.Tech CSE (AIML)
Year       : 1st SEMESTER
```

Figure 4. Search Student

```
=====
Enter your choice (1-9): 4

=====
                        UPDATE STUDENT
=====
Enter Student ID: 26BAI10807

Press ENTER to keep the existing value.
Enter new name [HARSH VARDHAN]: Harsh Vardhan
Enter new course [B.Tech CSE (AIML)]: AIML
Enter new year/semester [1st SEMESTER]: 1st

Student updated successfully!
```

Figure 5. Update Student

```
=====
Enter your choice (1-9): 5
=====
```

```
=====
                        DELETE STUDENT
=====
```

```
Enter Student ID: 26BAI10807
```

```
Student Details
=====
```

```
Student ID : 26BAI10807
Name       : Harsh Vardhan
Course    : AIML
```

```
Are you sure you want to delete this student? (y/n): y
```

```
Student deleted successfully!
```

Figure 6. Delete Student

```
=====
Enter your choice (1-9): 6
=====
```

```
=====
ENTER / UPDATE MARKS
=====
```

```
Enter Student ID: 26BAI10807
```

```
Student: HARSH VARDHAN
=====
```

```
Enter marks for Python (0-100): 99
```

```
Enter marks for Mathematics (0-100): 90
```

```
Enter marks for Computer Fundamentals (0-100): 95
```

```
Enter marks for Communication Skills (0-100): 100
```

```
Marks saved successfully!
```

Figure 7. Enter / Update Marks

```
=====
Enter your choice (1-9): 7
=====
```

```
=====
STUDENT REPORT
=====
```

```
Enter Student ID: 26BAI10807
```

```
=====
STUDENT PERFORMANCE REPORT
=====
```

```
Student ID : 26BAI10807
Name       : HARSH VARDHAN
Course    : B.Tech CSE (AIML)
Year      : 1st SEMESTER
```

```
Marks
=====
```

```
Python           : 99.00
Mathematics      : 90.00
Computer Fundamentals : 95.00
Communication Skills : 100.00
=====
```

```
Total Marks : 384.00
Percentage  : 96.00%
Grade       : A
Result      : Pass
Performance : Excellent Performance
=====
```

Figure 8. Student Performance Report

```
=====
Enter your choice (1-9): 8

=====
                        ALL PERFORMANCE REPORTS
=====
ID           Name           Percentage   Grade   Result
=====
26BAI10807  HARSH VARDHAN           96.00      A      Pass
=====
```

Figure 9. All Performance Reports

```
=====
                        STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Enter / Update Marks
7. Generate Student Report
8. Generate All Performance Reports
9. Exit
=====
Enter your choice (1-9): 9

=====
Thank you for using Student Management System!
Goodbye!
=====
harshvardhan@HARSHs-MacBook-Air / % █
```

Figure 10. Program Exit

03 • Conclusion

Final discussion of the project's implementation, learning outcomes, practical value and future development.

The **Student Management System using Python** successfully demonstrates how basic programming concepts can be combined to create a practical and meaningful application. The project was developed with the aim of simplifying student record management and academic performance tracking through a computerized system.

The application provides the essential operations required for managing student information. Users can add new students, view existing records, search for students, update their information and delete records when necessary. These operations provide the foundation of the system and demonstrate the basic principles of data management.

One of the most useful features of the application is its ability to manage academic marks. The system allows marks to be entered for Python, Mathematics, Computer Fundamentals and Communication Skills. It automatically calculates the total marks and percentage, assigns an appropriate grade and determines the student's result.

The use of automatic calculations reduces the possibility of manual arithmetic mistakes. Instead of calculating the total and percentage separately, the application performs these operations consistently every time. The grading system also follows predefined percentage ranges, making the evaluation process straightforward.

The project also gives importance to input validation. A program becomes much more useful when it can handle incorrect input instead of simply assuming that the user will always enter valid information. In this project, student IDs, names, courses and marks are checked before being accepted. Duplicate student IDs are prevented, and marks outside the range of 0 to 100 are rejected.

Error handling is another important part of the application. The system is designed to handle situations such as missing records, invalid numerical input and problems with stored data. This makes the program more reliable and provides the user with understandable feedback when something goes wrong.

JSON storage provides the application with persistent data management. Student records can be saved and loaded without requiring a separate database system. This keeps the project lightweight and easy to run while still demonstrating an important concept used in larger software applications.

The project also provides valuable learning experience from a programming perspective. While developing the system, concepts such as variables, data types, functions, loops, conditional statements, dictionaries, file handling, JSON and exception handling can be understood in a practical context.

The project demonstrates that programming is not only about writing individual pieces of code. A useful application requires planning, organization, validation, data management and proper handling of unexpected situations. Combining these elements into one project provides a much better understanding of how software is developed.

Another important outcome of the project is the understanding of CRUD operations. Create, Read, Update and Delete operations are present in many real-world applications, from simple record-management systems to large database-driven platforms. Implementing these operations in the Student Management System provides a basic understanding of how information is managed throughout its lifecycle.

The reporting feature also adds value to the system. Instead of simply storing marks, the application converts the stored information into a readable academic report. This demonstrates how software can transform raw data into useful information for users.

Although the current project is console-based, this should be viewed as the first stage of development rather than the final possible form of the system. A graphical interface could make the application more user-friendly. A database such as SQLite could provide more advanced storage capabilities. Authentication could allow different types of users to access different features.

Additional features such as attendance management, PDF report generation, data visualization and cloud storage could further increase the functionality of the system.

The project also provides an opportunity to understand the importance of writing maintainable code. As the application grows, dividing responsibilities into separate modules would make the program easier to understand, test and modify. The current implementation already separates many responsibilities through individual functions, providing a good foundation for future modularization.

From an academic perspective, the Student Management System is a suitable demonstration of first-semester programming knowledge because it does not depend on unnecessarily advanced technologies. Instead, it focuses on understanding fundamental programming concepts and applying them to a realistic problem.

The project also shows how a simple idea can be expanded gradually. A basic student record program can first support adding and viewing records. Then search, update and delete functionality can be added. After that, marks and performance calculations can be introduced. Finally, persistent storage, validation, reporting and error handling can make the system much more complete.

This gradual development process reflects an important lesson in software engineering: a good project does not have to begin as a complicated system. It can begin with a clear problem and grow through well-defined improvements.

In conclusion, the **Student Management System using Python** achieves its primary objective of providing a simple and organized method of managing student information and academic performance. It combines student record management, CRUD operations, marks processing, performance calculation, reporting, JSON storage, validation and error handling into a single application.

More importantly, the project has helped connect theoretical Python concepts with practical software development. It provides a foundation for understanding how programs are structured, how data is managed, how errors are handled and how user requirements can be converted into working functionality.

The current system can serve as the foundation for a much larger application in the future. With the addition of a graphical interface, database support, authentication, attendance management, analytics and cloud capabilities, it could eventually become a complete academic management platform.

For a first-semester B.Tech CSE/AI ML project, the Student Management System provides a practical demonstration of programming fundamentals while leaving significant room for future improvement and experimentation. It is therefore not only a completed academic project but also a starting point for learning how simple Python programs can gradually evolve into more advanced software systems.

End of Project Report